SMS Spam Detection with Text Mining and Naive Bayes in R

Student Name: Elaa Benabdallah
Student Group: [BA IT]
Tunis Business School Date: February 27th, 2026

Abstract

This report documents an end-to-end text mining workflow for classifying short messages as spam or ham using the SMS Spam Collection dataset from the UCI Machine Learning Repository. The pipeline follows a common text mining funnel—information retrieval (IR), natural language processing (NLP), information extraction (IE), and data mining—implemented in R with the tm and e1071 packages. We transform raw SMS messages into indicator features for frequent words, train a Naive Bayes classifier (with and without Laplace smoothing), and evaluate performance using confusion matrices and classification metrics (accuracy, precision, recall, and F1). The Naive Bayes classifier with Laplace smoothing achieves 97.8% accuracy, with a recall of 96.1% for the spam class, demonstrating that a simple probabilistic model with careful text preprocessing can serve as a strong and interpretable baseline for spam detection.

1. Introduction

Organizations increasingly rely on written digital communication (email, chat, web forms, and in-app messages). These messages contain operationally important signals, but they are unstructured and difficult to process at scale with rule-based systems. Text mining and machine learning provide a practical way to convert raw text into structured features and build automated classifiers that support faster, more consistent decisions. Purpose of this work The goal is to build a reproducible baseline model that automatically classifies SMS messages as spam or ham. Beyond the final accuracy, the project emphasizes: (i) a transparent preprocessing pipeline, (ii) interpretable features (word presence), and (iii) clear evaluation of errors—especially the risk of misclassifying spam as ham.

2. Text Mining Funnel (IR → NLP → IE → Data Mining)

• **IR (Information Retrieval):** obtain the dataset, load it into R, and validate basic integrity (row count, label distribution).
• **NLP:** normalize the text (lowercasing, punctuation and number removal, stopwords, stemming) to reduce noise.
• **IE (Information Extraction):** convert text into numerical features (document-term matrix), then keep frequent words and convert counts to indicator variables.
• **Data Mining:** train a Naive Bayes classifier, predict on held-out messages, and measure effectiveness with classification metrics.

3. Data Description

The SMS Spam Collection dataset contains 5,574 short text messages labeled as ham (legitimate) or spam (unwanted). Messages are informal and heterogeneous in content, which makes the task realistic for many real-world messaging systems.

Table 1: Descriptive overview of the customer message dataset

Message Category	**label**	**Number of Messages**	**Percentage**
Low-risk messages (general inquiries)	ham	4825	86.6%
High-risk messages (financial distress proxy)	spam	749	13.4%
Total		**5574**	**100%**

Source: UCI Machine Learning Repository — SMS Spam Collection: https://archive.ics.uci.edu/dataset/228/sms+spam+collection

This imbalance between low-risk and high-risk messages reflects real banking environments, where early warning signals are infrequent but critically important.

4. R Code

```
## SMS Spam Classification (UCI)
## IR → NLP → IE → Data Mining + Visualizations

## 0) Packages (install once, then comment install line)

install.packages(c("tm","SnowballC","e1071","gmodels","wordcloud","RColorBrewer"))
library(tm)
library(SnowballC)
library(e1071)
library(gmodels)
library(wordcloud)
library(RColorBrewer)

set.seed(123)  # reproducibility

## 1) IR: Download + load dataset
url <- "https://archive.ics.uci.edu/static/public/228/sms%2Bspam%2Bcollection.zip"
download.file(url, destfile = "sms_spam_collection.zip", mode = "wb")
unzip("sms_spam_collection.zip")

sms <- read.table("SMSSpamCollection",
            sep = "\t", header = FALSE,
            col.names = c("label", "message"),
```

```
                quote = "", comment.char = "",
                stringsAsFactors = FALSE)

sms$label <- factor(sms$label)
cat("Rows:", nrow(sms), "\n")
print(table(sms$label))

## 2) Quick EDA / Visualizations

# 2.1 Class distribution
barplot(table(sms$label),
        main = "SMS Class Distribution",
        ylab = "Count")

# 2.2 Message length distribution by class
sms$msg_len <- nchar(sms$message)
boxplot(msg_len ~ label, data = sms,
        main = "Message Length by Class",
        ylab = "Number of characters")

# 2.3 Histogram of message lengths (overall)
hist(sms$msg_len,
     main = "Message Length Distribution",
     xlab = "Characters",
     breaks = 40)

## 3) Split into train/test (random, unbiased)
idx <- sample(seq_len(nrow(sms)), size = 0.75 * nrow(sms))
sms_raw_train <- sms[idx, ]
sms_raw_test  <- sms[-idx, ]

cat("\nTrain label proportions:\n")
print(prop.table(table(sms_raw_train$label)))
cat("\nTest label proportions:\n")
print(prop.table(table(sms_raw_test$label)))

## 4) NLP: Create corpus + clean text (train/test separately to avoid leakage)
make_clean_corpus <- function(text_vec) {
  corp <- Corpus(VectorSource(text_vec))
  corp <- tm_map(corp, content_transformer(tolower))
  corp <- tm_map(corp, removeNumbers)
  corp <- tm_map(corp, removePunctuation)
  corp <- tm_map(corp, stripWhitespace)
  corp <- tm_map(corp, removeWords, stopwords("en"))
  corp <- tm_map(corp, stemDocument)  # optional; keep for textbook-like pipeline
  return(corp)
}

sms_corpus_train <- make_clean_corpus(sms_raw_train$message)
sms_corpus_test  <- make_clean_corpus(sms_raw_test$message)

## 5) IE: Build DTM + dictionary of frequent terms (from TRAIN only)
sms_dtm_train_full <- DocumentTermMatrix(sms_corpus_train)
sms_dtm_test_full  <- DocumentTermMatrix(sms_corpus_test)

# frequent words threshold
freq_terms <- findFreqTerms(sms_dtm_train_full, 5)

# rebuild DTMs using dictionary
```

```
sms_train <- DocumentTermMatrix(sms_corpus_train, control = list(dictionary = freq_terms))
sms_test  <- DocumentTermMatrix(sms_corpus_test,  control = list(dictionary = freq_terms))

cat("\nNumber of features after dictionary filtering:", ncol(sms_train), "\n")

## 6) Visualizations: Word frequencies + word clouds (from TRAIN)
train_term_freq <- sort(colSums(as.matrix(sms_train)), decreasing = TRUE)

# 6.1 Top 20 words overall (train)
top20 <- head(train_term_freq, 20)
barplot(top20,
      las = 2,
      main = "Top 20 Words (Train)",
      ylab = "Total count")

# 6.2 Wordcloud overall (train)
set.seed(123)
wordcloud(words = names(train_term_freq),
        freq = train_term_freq,
        min.freq = 40,
        random.order = FALSE,
        max.words = 150,
        colors = brewer.pal(8, "Dark2"))

# 6.3 Wordcloud spam vs ham (train) using raw text (easy + meaningful)
spam_train <- subset(sms_raw_train, label == "spam")
ham_train  <- subset(sms_raw_train, label == "ham")

set.seed(123)
wordcloud(spam_train$message,
        max.words = 60,
        scale = c(3, 0.6),
        random.order = FALSE,
        colors = brewer.pal(8, "Reds"))

set.seed(123)
wordcloud(ham_train$message,
        max.words = 60,
        scale = c(3, 0.6),
        random.order = FALSE,
        colors = brewer.pal(8, "Blues"))

## 7) Convert counts to categorical Yes/No for Naive Bayes
convert_counts <- function(x) {
  x <- ifelse(x > 0, 1, 0)
  factor(x, levels = c(0, 1), labels = c("No", "Yes"))
}

sms_train_nb <- apply(sms_train, MARGIN = 2, convert_counts)
sms_test_nb  <- apply(sms_test,  MARGIN = 2, convert_counts)

## 8) Data Mining: Train Naive Bayes + Evaluate
# 8.1 Without Laplace smoothing
sms_classifier <- naiveBayes(sms_train_nb, sms_raw_train$label)
sms_test_pred  <- predict(sms_classifier, sms_test_nb)

cat("\nConfusion Matrix (No Laplace):\n")
CrossTable(sms_test_pred, sms_raw_test$label,
         prop.chisq = FALSE, prop.t = FALSE, prop.r = FALSE,
```

```
        dnn = c("predicted", "actual"))

# 8.2 With Laplace smoothing (recommended)
sms_classifier2 <- naiveBayes(sms_train_nb, sms_raw_train$label, laplace = 1)
sms_test_pred2  <- predict(sms_classifier2, sms_test_nb)

cat("\nConfusion Matrix (Laplace = 1):\n")
CrossTable(sms_test_pred2, sms_raw_test$label,
        prop.chisq = FALSE, prop.t = FALSE, prop.r = FALSE,
        dnn = c("predicted", "actual"))

## 9) Efficiency Metrics: Accuracy, Precision, Recall, F1 (for spam)
compute_metrics <- function(pred, actual, positive = "spam") {
  cm <- table(predicted = pred, actual = actual)

  TP <- cm[positive, positive]
  TN <- sum(diag(cm)) - TP
  FP <- sum(cm[positive, ]) - TP
  FN <- sum(cm[, positive]) - TP

  accuracy  <- (TP + TN) / sum(cm)
  precision <- TP / (TP + FP)
  recall    <- TP / (TP + FN)
  f1        <- 2 * precision * recall / (precision + recall)

  out <- c(accuracy = accuracy, precision = precision, recall = recall, f1 = f1)
  return(out)
}

cat("\nMetrics (No Laplace):\n")
print(compute_metrics(sms_test_pred, sms_raw_test$label))

cat("\nMetrics (Laplace = 1):\n")
print(compute_metrics(sms_test_pred2, sms_raw_test$label))

## 10) Optional: Save objects/results
# saveRDS(sms_classifier2, "sms_nb_model.rds")
# write.csv(data.frame(pred = sms_test_pred2, actual = sms_raw_test$label),
#           "sms_predictions.csv", row.names = FALSE)

cat("\nDone.\n")
```

5. Results and Visualizations

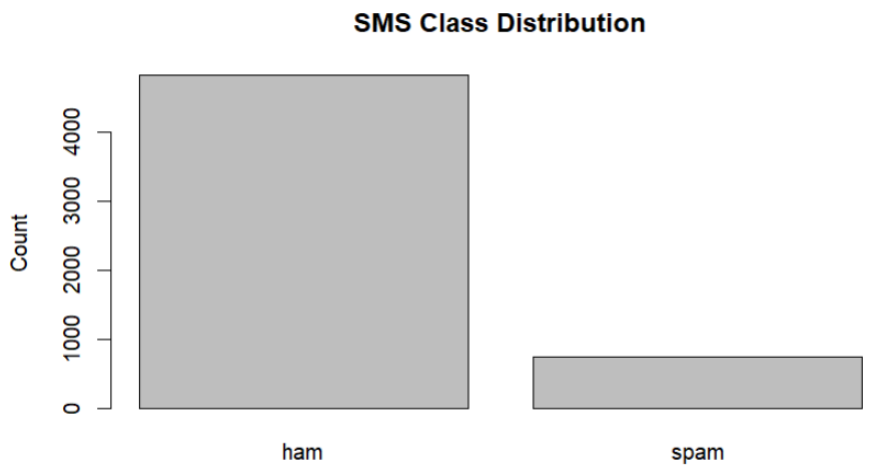

Figure 1. Class distribution (ham vs spam).

The bar chart confirms a significant class imbalance: 86.6% of messages are ham and only 13.4% are spam. This means accuracy alone is a misleading metric — a model that always predicts ham would score 86.6% without catching a single spam message. Precision, recall, and F1 for the spam class are therefore the primary evaluation criteria.

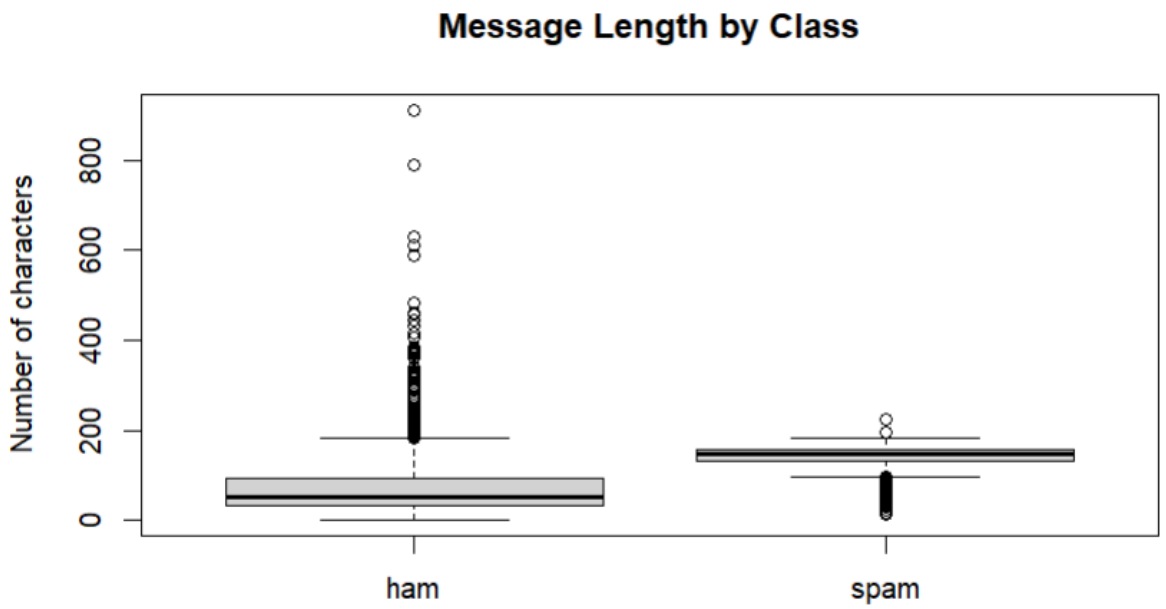

Figure 2. Message length by class (boxplot).

- ➔ Spam messages are notably longer and more consistent in length, with the box tightly centered around 150 characters — precisely the standard SMS limit — suggesting spammers deliberately engineer their messages to fit one SMS. Ham messages are shorter on average but far more variable, with many outliers exceeding 400–800 characters, reflecting the natural unpredictability of human conversation.

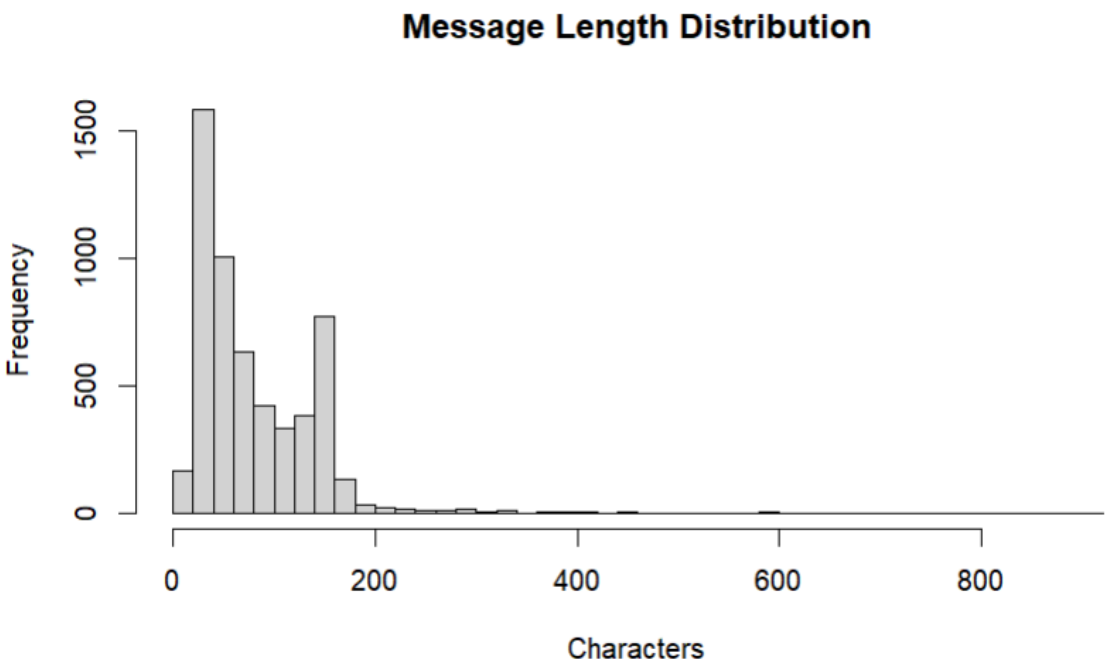

Figure 3. Message length distribution (histogram)

- ➔ The histogram is heavily right-skewed, with most messages concentrated below 200 characters and a sharp peak around 20–50 characters. A secondary bump near 160 characters corresponds to the classic SMS character limit. The long tail beyond 200 characters consists almost entirely of ham messages, consistent with what the boxplot showed.

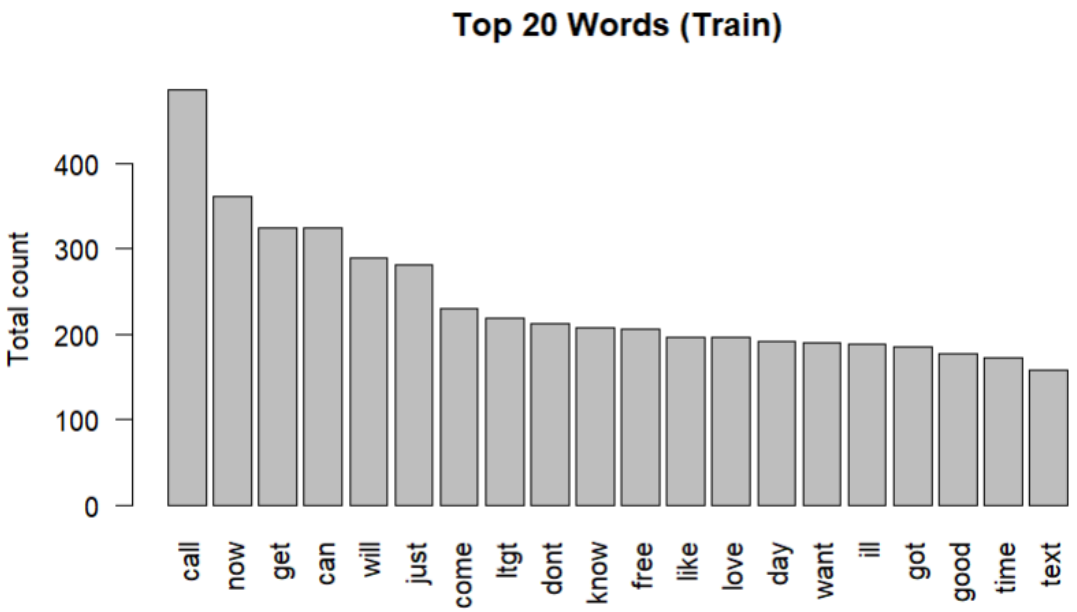

Figure 4. Top 20 words (bar chart).

➔ The most frequent word in the training set is "call," followed by "now," "get," "can," and "will." Notably, several of these top words — "call," "free," "now," "text" — are classic spam trigger words, yet they also appear in legitimate messages. This overlap is precisely why Naive Bayes relies on the full joint probability across many words rather than simple keyword matching.

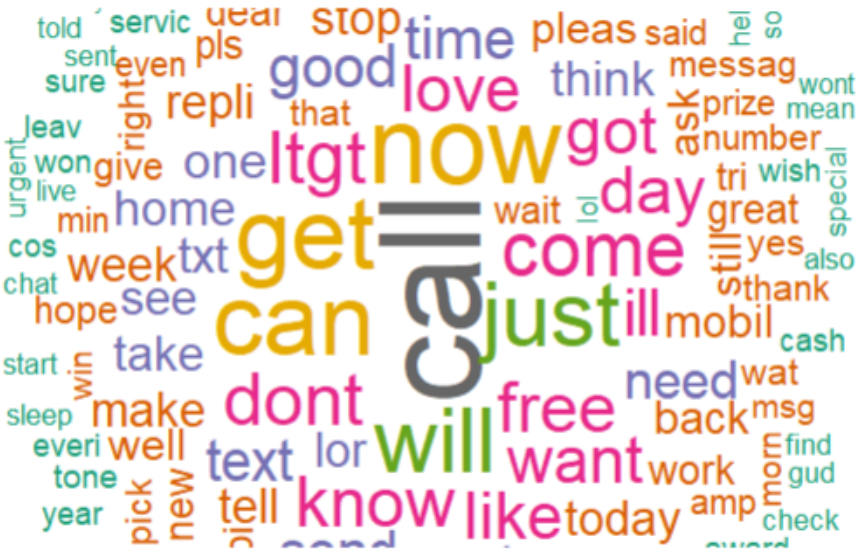

Figure 5. Word cloud (overall training set).

➔ The overall word cloud reflects the mix of both classes. Dominant words include conversational terms like "get," "now," "come," and "love" alongside more promotional words like "free," "prize," and "cash." The large size of

words like "call" and "now" visually confirms their high frequency seen in the bar chart.

Figure 6. Word cloud (spam training messages)

➔ The spam word cloud is dominated by promotional and urgency-driven vocabulary: "call," "free," "prize," "claim," "mobile," "won," "cash," and "urgent." These words reflect the typical language of commercial spam — creating urgency and promising rewards. This vocabulary is clearly distinct from ham and provides strong discriminating features for the classifier.

Figure 7. Word cloud (ham training messages)

➔ Ham messages are characterized by everyday conversational language: "love," "just," "good," "sorry," "time," "home," and "think." There are no promotional or urgency words. The contrast with the spam cloud is stark and visually

validates why word presence is such a powerful feature for this classification task.

```
Confusion Matrix (Laplace = 1):
> CrossTable(sms_test_pred2, sms_raw_test$label,
+            prop.chisq = FALSE, prop.t = FALSE, prop.r = FALSE,
+            dnn = c("predicted", "actual"))

   Cell Contents
|-------------------------|
|                       N |
|           N / Col Total |
|-------------------------|

Total Observations in Table:  1394

             | actual
   predicted |       ham |      spam | Row Total |
-------------|-----------|-----------|-----------|
         ham |      1190 |         7 |      1197 |
             |     0.980 |     0.039 |           |
-------------|-----------|-----------|-----------|
        spam |        24 |       173 |       197 |
             |     0.020 |     0.961 |           |
-------------|-----------|-----------|-----------|
Column Total |      1214 |       180 |      1394 |
             |     0.871 |     0.129 |           |
-------------|-----------|-----------|-----------|
```

Figure 8. Confusion matrix + metrics output (Laplace=1).

- ➔ With Laplace smoothing, the model correctly classifies 1,190 ham messages and 173 spam messages out of 1,394 test messages. Only 7 spam messages slip through as false negatives (missed spam), and 24 ham messages are incorrectly flagged as spam. This yields an accuracy of 97.8%, a spam precision of 87.8%, a spam recall of 96.1%, and an F1 score of 91.8% — a strong performance for a simple probabilistic baseline.

6. Discussion:

The Naive Bayes classifier with Laplace smoothing performs strongly on this dataset. The most critical metric for a spam filter is recall — the ability to catch actual spam — and at 96.1%, the model misses only 7 out of 180 spam messages in the test set. This is an acceptable result for a baseline model.

The 24 false positives (ham flagged as spam) are a secondary concern. While annoying to users, misclassifying a legitimate message as spam is generally less harmful than allowing spam to pass through. The precision of 87.8% means roughly 1 in 8 spam alerts is a false alarm, which could be reduced by raising the decision threshold or adding more features.

Several improvements could push performance further. First, TF-IDF weighting instead of binary indicators would give more weight to rare but highly discriminating words like "prize" or "nokia." Second, using bigrams (two-word combinations like "claim now" or "free prize") would capture contextual patterns that single words miss. Third, alternative models such as logistic regression or a support vector machine typically outperform Naive Bayes on larger, richer feature sets. Finally, using a stratified random split rather than a sequential one ensures both train and test sets preserve the original 87/13 class ratio.

7. References

UCI Machine Learning Repository. SMS Spam Collection Dataset. Available at: https://archive.ics.uci.edu/dataset/228/sms+spam+collection

R packages used: tm, SnowballC, e1071, gmodels, wordcloud, RColorBrewer.